

Practical Privacy-Preserving Convolutional Neural Network Inference Framework With Edge Computing for Health Monitoring

Ruoli Zhao¹, Yong Xie¹, Debiao He¹, *Member, IEEE*, Kim-Kwang Raymond Choo², *Senior Member, IEEE*, and Zoe L. Jiang³

Abstract—Using Convolutional Neural Network (CNN) model to analyze monitoring data in Body Area Network (BAN) has become an important way to solve health related issues in the current large sub-health population and aging population. However, the inference and analysis process of BAN data needs to ensure efficiency and security. At present, ensuring a balance of efficiency and security in the inference of CCN models is challenging. Therefore, an efficient and secure CNN inference scheme is proposed based on two Edge-Cloud-Servers (CS_0 and CS_1). By analyzing the CNN model and combining two secret sharing semantics, we optimize the communication overhead of inference. Specifically, a new non-interactive secure convolutional layer computation protocol is designed to significantly reduce the number of interactions between CS_0 and CS_1 . For non-linear layers, we propose a simpler secure comparison computation functionality to reduce the communication overhead. Moreover, we also design some lightweight secure building blocks based on secret sharing to improve computing efficiency. We implement our proposed scheme on two standard datasets. Through the theoretical analysis and experimental comparison, our scheme improves the computational efficiency.

Index Terms—Privacy-preserving, secret sharing, CNN, secure inference, edge computing.

I. INTRODUCTION

WITH the further development of economic globalization, people's work pressure is increasing and the pace of life is becoming faster and faster, which leads to many people's health in a sub-health state [1]. Every year, many people in the world miss the best treatment time because they don't have time to see a doctor, which leads to minor illness gradually becoming serious illness. To avoid the problem, deep learning has been widely applied in health monitoring [2], [3] and brings a lot of convenience to people's lives. Therefore, the health monitoring oriented BAN can analyze the users' health data using deep learning. For example, many wireless BAN devices are used in the remote wireless monitoring system for the elderly, so that doctors or family members can timely grasp the physical condition of the elderly. A common BAN application scenario is shown in Fig. 1. However, the very valuable BAN has not been widely used. The most critical reason is that the BAN and its data analysis network involve people's most private data. Once the privacy data is leaked, it may cause many unexpected consequences, such as rampant medical promotion, malicious personal attacks, loss of economic benefits, company bankruptcy, group events, political chaos, etc.

It can be seen that it is extremely important for the security of BAN used for health monitoring. In order to meet the crying need, many researchers have researched the security of the inference process [4], [5], [6]. The goals of privacy-preserving are the data of BAN devices, inference results and model parameters of the model owner. In order to realize the secure inference process, many privacy-preserving technologies have become the effective methods [7], [8]. For the Differential Privacy (DP), DP can well protect privacy, but it has a great impact on inference accuracy. The combination of machine learning model and cryptographic technologies (Homomorphic Encryption, HE and Secure Multi-party Computation, SMC) makes the overhead of BAN devices and model owner very large [9], [10]. Therefore, many schemes outsource the computation process to cloud servers [11], [12]. However, the inference efficiency is not satisfactory due to the complex cryptographic operations.

Manuscript received 16 October 2023; revised 13 April 2024; accepted 20 July 2024. Date of publication 2 August 2024; date of current version 15 November 2024. The work of Kim-Kwang Raymond Choo was supported only by the Cloud Technology Endowed Professorship. This work was supported in part by the National Natural Science Foundation of China under Grant 62462053, in part by the Science and Technology Foundation of Qinghai Province under Grant 2023-ZJ-731, in part by the Open Project of the Qinghai Provincial Key Laboratory of Restoration Ecology in Cold Area under Grant 2023-KF-12, in part by the Shenzhen Science and Technology Major Special Project under Grant KJZD20230923114608017, and in part by the Open Research Fund of Guangdong Key Laboratory of Blockchain Security, Guangzhou University. Recommended for acceptance by Dr. Dejun Yang. (*Corresponding author: Yong Xie.*)

Ruoli Zhao is with the School of Computer Technology and Applications, Qinghai University, Xining 810016, China (e-mail: zhaoruoli@126.com).

Yong Xie is with the School of Computer Technology and Applications, Qinghai University, Xining 810016, China, also with the School of Computer and Information Science, Qinghai Institute of Technology, Xining 810016, China, and also with the Guangdong Key Laboratory of Blockchain Security, Guangzhou University, Guangzhou 511370, China (e-mail: mark.y.xie@qq.com).

Debiao He is with the Key Laboratory of Aerospace Information Security and Trusted Computing, Ministry of Education, Wuhan 430072, China, and also with the School of Cyber Science and Engineering, Wuhan University, Wuhan 430072, China (e-mail: hedebiao@163.com).

Kim-Kwang Raymond Choo is with the Department of Information Systems and Cyber Security, University of Texas at San Antonio, San Antonio, TX 78249 USA (e-mail: Raymond.Cho@fulbrightmail.org).

Zoe L. Jiang is with the School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen 518055, China, and also with the Pengcheng Laboratory, Shenzhen 518055, China (e-mail: zoeljiang@hit.edu.cn).

Digital Object Identifier 10.1109/TNSE.2024.3434643

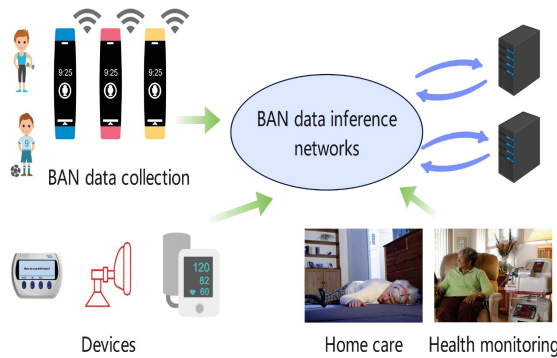


Fig. 1. BAN architecture.

To improve the efficiency of secure inference, researchers mainly focus on HE and SMC. There are two major problems in improving the efficiency of HE. First, the inference process requires high computation resources of the participants, which will lead to excessive computation overhead. Second, homomorphic computation is difficult to realize complex non-linear operations in some machine learning models, such as ReLU and Max Pooling in CNN [13]. Some researchers also propose to use polynomials to complete nonlinear operations in homomorphic computation, but the method will reduce the accuracy of model inference [14]. Therefore, applying SMC to implement the secure inference is becoming more popular [10], [15]. Liu et al. [16] first propose the concept of Oblivious Neural Network (ONN), which refers to achieving efficient and reasonable privacy protection without modifying the neural network model. Li et al. [17] design a secure CNN inference scheme with two servers. They design the secure ReLU computation with garbled circuits, which brings huge communication overhead. At the same time, some researchers have proposed some general SMC frameworks that can achieve privacy-preserving inference. In [18], the framework uses two servers setting. Some frameworks use three or more servers setting [19], [20], which is not easy to deploy in the actual environment. However, the existing general frameworks are difficult to meet the computational efficiency requirements of the edge computing scenario. To adapt the need of edge computing scenario, Wang et al. [4] propose a new framework for the secure CNN inference. However, the interactions of their proposed framework are very large, which makes the scheme not practical.

A. Our Contributions

Based on the above analysis, the existing schemes bring huge communication overhead for the participants. Therefore, a privacy-preserving and efficient CNN inference framework for the resource-constrained edge cloud servers is proposed in this paper. The contributions are as follows.

- 1) We propose a secure CNN inference framework. Specifically, we use two edge cloud servers to complete the secure inference. By combining two secret sharing semantics, we significantly optimize the communication overhead in the secure CNN inference process.

- 2) We design a new non-interactive secure convolutional layer computation protocol, which greatly reduced the number of interactions between two edge cloud servers. A simpler secure comparison functionality is designed to reduce the number of interactions required by ReLU and Max pooling. The characteristics of low rounds and low communication overhead can significantly improve the computational efficiency.
- 3) Some lightweight secure computation protocols are designed to construct a secure CNN inference protocol and achieve a real-time inference process, including secure comparison, secure linear layer, secure ReLU, secure pooling layer and secure conversion.
- 4) The experimental evaluations on realistic datasets show that our proposed framework is efficient. To make a complete comparison, we implement our framework and the compared schemes. Through the comparison results, our proposed framework give less burden to the edge cloud servers.

B. Organization of the Paper

The related work is introduced in Section II. Then, we introduce the preliminaries in Section III. The system overview is described in Section IV. The building blocks and proposed framework are given in Section V and Section VI respectively. We give the security analysis in Section VII. In Section VIII, we give the performance evaluation. Finally, we conclude this paper in Section IX.

II. RELATED WORK

Health prediction based on CNN model has been widely used in BAN devices [21]. Specifically, the BAN data is sent to a prediction service center. The prediction service center use its own CNN model to predict the BAN data. Then, the results will be returned to the BAN devices. Although the above scenario has great practical application value, there is also a serious privacy disclosure problem. Therefore, the research of secure machine learning inference scheme has become a hot issue. At present, many schemes are based on HE or SMC to achieve data privacy protection.

HE enables the participants to directly compute the encrypted data. Therefore, many secure machine learning inference schemes are designed based on HE. Xie et al. [22] propose a secure diagnosis scheme and design some secure computation protocols based on HE. However, the application of Paillier algorithm results in a higher computational time and communication overhead for secure prediction. Li et al. [23] propose a verifiable and secure inference scheme based on OU homomorphic encryption. Through their scheme, the input data, model parameters and prediction results will be protected. Although the authors design a verify method to prevent the results from being tampered, the method is only applied to a simple linear regression model. Based on the Paillier encryption algorithm, Zhao et al. [24] design a secure outsourcing computation toolkit, which implements multiple secure operators to support basic

computation. However, due to using threshold encryption and decryption, the communication overhead is high.

SMC is performed by two or more participants. In [25], Mohassel et al. propose a general framework for privacy-preserving machine learning in a three-server setting. The authors design a new truncation protocol, which is suitable for secure training and prediction. Byali et al. [26] propose an efficient four-party framework and the framework can achieve the secure server-aided prediction. Their scheme can enable honest participants to obtain computational results no matter what type of attack carried out by malicious adversaries. Dalskov et al. [27] propose a novel four-party honest-majority privacy preserving computation protocol, which can achieve the active security. Huang et al. [28] propose a framework for secure CNN feature extraction based on edge computing. Although the scheme in [28] is efficient, the authors assume that the CNN model is public.

III. PRELIMINARIES

A. Secret Sharing

We use secret sharing technique to protect the data privacy in this paper.

Arithmetic sharing: Based on [29], we define two sharing semantics as $[\cdot]^A$ and $\langle \cdot \rangle$.

In $[\cdot]^A$ -**sharing**, a value $x \in Z_{2^l}$ is shared between P_0 and P_1 such that $x \equiv [x]_0^A + [x]_1^A \pmod{2^l}$. P_i ($i \in \{0, 1\}$) holds $[x]_i^A$. It should be noted that all operations on the ring 2^l require modulus. For simplicity, we omit the modulus symbol ($\pmod{2^l}$) in the subsequent descriptions. Given $[x]^A$ and $[y]^A$, the addition/subtraction and scalar multiplication can be computed by P_i locally such that $[z]_i^A = [x]_i^A \pm [y]_i^A$, $[y]_i^A = t \cdot [x]_i^A$ (t is constant value). The multiplication $[z]^A = [x]^A \cdot [y]^A$ is completed with the Beaver's triplet (a, b, c) [3], where $c = a \cdot b$. Specifically, P_i computes $[e]_i^A = [x]_i^A - [a]_i^A$, $[f]_i^A = [y]_i^A - [b]_i^A$ and sends them to P_{1-i} . Then, P_i computes $[z]_i^A = i \cdot e \cdot f + f \cdot [a]_i^A + e \cdot [b]_i^A + [c]_i^A$.

In $\langle \cdot \rangle$ -**sharing**, we assume that there exist values U_x, δ_x and the values satisfy $U_x = x + \delta_x$ for a value $x \in Z_{2^l}$. Given $\langle x \rangle$ and $\langle y \rangle$, the addition/subtraction and scalar multiplication can be computed by P_i locally such that $\langle z \rangle_i = (U_z, [\delta_z]_i^A)$ ($U_z = t_1 \cdot U_x + t_2 \cdot U_y$ and $[\delta_z]_i^A = t_1 \cdot [\delta_x]_i^A + t_2 \cdot [\delta_y]_i^A$). To complete multiplication $\langle z \rangle = \langle x \rangle \cdot \langle y \rangle$, P_i computes $[U_z]_i^A = i \cdot U_x U_y - U_x [\delta_y]_i^A - U_y [\delta_x]_i^A + [\delta_{x,y}]_i^A + [\delta_z]_i^A$ and sends to P_{1-i} . P_i will obtain U_z and holds $\langle z \rangle_i = (U_z, [\delta_z]_i^A)$. It should be noted that $[\delta_{x,y}]_i^A$ can be generated in the offline phase.

Boolean sharing: A bit $x \in Z_2$ is shared between P_0 and P_1 such that $x = [x]_0^B \oplus [x]_1^B$. P_i holds $[x]_i^B$. It should be noted that the addition and multiplication operations in $[\cdot]^A$ -**sharing** are replaced with XOR ($\oplus$) and AND ($\wedge$) respectively.

B. Convolutional Neural Network

CNN model is generally composed of convolutional layer, normalization, ReLU function, pooling layer and fully-connected layer.

Convolutional Layer: The convolutional layer can extract the features of a local region. Different convolutional kernels are

equivalent to different feature extractors. The computation process of convolutional layer can be transformed to multiplication and addition operations.

Normalization: After the convolutional computation is completed, the data should be normalized to a standard distribution within a fixed interval range.

ReLU Function: ReLU is an activation function, which can enhance the representation ability and learning ability of the network. The ReLU function is expressed as $ReLU(x) = \max(0, x)$. The input of ReLU function is the convolution values.

Pooling Layer: The pooling layer can select features and reduce the number of features. The pooling operation refers to the down sampling of each region to obtain a value as a summary of the region. There are two common pooling functions, average pooling and max pooling.

Fully-Connected Layer: The computation of fully-connected layer can be transformed to multiplication and addition operations, which is the same as convolutional layer.

IV. SYSTEM OVERVIEW

A. System Model

In Fig. 2, we show the system model. It should be noted that we omit Parameter Server in Fig. 2.

There are five participants in the proposed system, which are Parameter Server (PS), BAN devices (BD), two edge cloud servers (CS_0, CS_1), Service Provider (SP).

- *Parameter Server (PS):* PS is a third party. PS generates system parameters and send them to BD, CS_i and SP by a secure communication channel. PS does not participant in the online phase.
- *BAN devices (BD):* The BAN devices have the unclassified data. Each BD will send query request to CS_0, CS_1 and obtains the inference results. In the inference phase, the BD does not need to participate in the computation.
- *Edge Cloud Servers (CS_0, CS_1):* In our system model, CS_0 and CS_1 are two non-colluding edge cloud servers, which will complete secure inference with CNN model after receiving the request of BD. Then, CS_0 and CS_1 will send the computation results to BD secretly.
- *Service Provider (SP):* The SP owns a trained CNN model. Because of the limited computing and communication resources, SP chooses to outsource the inference service to the edge cloud servers.

B. Threat Model and Security

In this paper, the security is defined in the simulation paradigm [30], [31], [32] against a static semi-honest probabilistic polynomial time adversary. In the two-party computing scenario, a computationally bounded adversary corrupts either P_0 or P_1 at the beginning of the protocol.

Specifically, we use ideal-real simulation paradigm to prove the security of two-party protocol. In the ideal world, each party sends the inputs to a functionality and the functionality is trusted. Then, the trusted functionality completes the computation and sends the outputs to the parties respectively. In the real world,

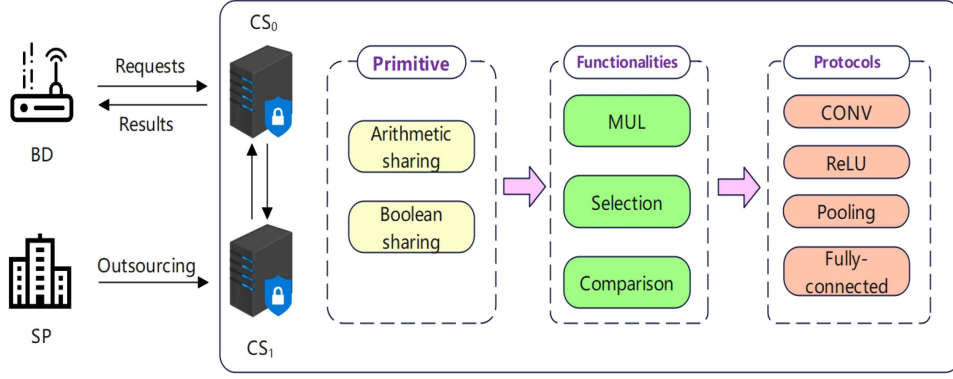


Fig. 2. System model.

TABLE I
NOTATION AND DEFINITION

Notation	Definition
l	the bit length
$P_i \in \{0, 1\}$	the participants of secure computation
$[x]_i^A$	$[\cdot]^A$ -sharing of x and $x = [x]_0^A + [x]_1^A$
$\langle x \rangle_i = (U, [\delta]_i^A)$	$\langle \cdot \rangle$ -sharing of x and $x = U - \delta$
$[x]_i^B$	boolean sharing of $x \in \mathbb{Z}_2$ and $x = [x]_0^B \oplus [x]_1^B$
$\mathcal{F}$	functionality
Π	two-party protocol
$k_{\{0,1\}}$	key of pseudorandom function between P_0 and P_1
$\lfloor x \rfloor$	the round-down operation
$\mathbf{W}$	CNN model parameter
$R^{U \cdot V \cdot D \cdot P}$	the dimension of convolutional filter
$\mathbf{X}$	the input feature image
$R^{M \cdot N \cdot D}$	the dimension of input image
$T(\cdot)$	the truncation operation

TABLE II
FUNCTIONALITIES

Functionality	Propose	input	Applying for
$\mathcal{F}_{3-mult}$	[29]	$\langle a \rangle, \langle b \rangle, \langle c \rangle$	Linear layer
$\mathcal{F}_S$	[4]	$[s]_i^B, [x]_i^A, [y]_i^A$	ReLU, Max pooling
$\mathcal{F}_{SC}$	Ours	$[x]_i^A$	ReLU, Max pooling

there is no trusted functionality. The parties complete the computation with interactions. If there is an adversary in the real world, there exists a corresponding adversary (called the simulator) in the ideal world. Meanwhile, the views (including inputs, outputs and intermediate values) in the ideal world and real world are computationally indistinguishable. Therefore, it indicates that the protocol execution in the real world has successfully simulated the protocol execution in the ideal world, which proves that the protocol is secure [33].

In our proposed scheme, we assume that the CS_0 and CS_1 are non-colluding. The assumption is reasonable because CS_0 and CS_1 belong to different companies. These two competing companies not collude with each other to steal sensitive data for the reputation. At the same time, there are two security threats. On the one hand, the adversary (SP or CS_i) will try to learn the data of BD. On the other hand, the adversary (BD or CS_i) will try to learn the model of SP.

V. BUILDING BLOCKS

In this section, we construct a series of building blocks for the secure CNN inference. The used notations are shown in Table I. We divide this section into two parts. The first part is functionalities and the second part is protocols. Specifically, we

start by describing three functionalities for subsequent computation. These functionalities are used for the design of the protocols. Then, we design some secure computation protocols for each layer based on the layers' characteristics of the CNN model, including convolutional layer, fully-connected layer, ReLU and pooling layer.

A. Functionalities

In Table II, we list the functionalities applied in our proposed protocols.

1) *3-Input Multiplication*: The 3-input multiplication functionality is proposed in [29]. Given the $\langle \cdot \rangle$ -sharing of a , b and c . The functionality computes $[U_y]^A$ where $U_y = a \cdot b \cdot c$. We employ the functionality for the convolutional layer, which computes the convolution operation and normalization operation simultaneously. The functionality $\mathcal{F}_{3-mult}(\langle a \rangle, \langle b \rangle, \langle c \rangle)$ is described as follows.

- 1) P_i holds $\langle a \rangle_i, \langle b \rangle_i, \langle c \rangle_i$.
- 2) P_i computes $[y]_i^A = i \cdot U_{abc} - U_{ab}[\delta_c]_i^A - U_{bc}[\delta_a]_i^A - U_{ac}[\delta_b]_i^A + U_a[\delta_{bc}]_i^A + U_b[\delta_{ac}]_i^A + U_c[\delta_{ab}]_i^A - [\delta_{abc}]_i^A$.
- 3) In (2), the terms $[\delta_{bc}]_i^A, [\delta_{ac}]_i^A, [\delta_{ab}]_i^A, [\delta_{abc}]_i^A$ can be generated by PS in the offline phase.

2) *Secure Selection*: The secure selection is applying for ReLU and max pooling. Given a boolean sharing of s (if $x - y > 0$, $s = 1$, else $s = 0$) and $[x]_i^A, [y]_i^A$, if $s = 1$ P_i selects $[x]_i^A$, else $[y]_i^A$. The functionality $\mathcal{F}_S([s]_i^B, [x]_i^A, [y]_i^A)$ is proposed in [4].

- 1) P_i holds $[s]_i^B, [x]_i^A$ and $[y]_i^A$.
- 2) P_i inputs $((-1)^{[s]_i^B} [x - y]_i^A, (-1)^{[s]_i^B})$ and performs the multiplication with Beaver's triplets. The Beaver's triplets are generated by PS in the offline phase.
- 3) P_i obtains the multiplication result $[\delta]_i^A$ and computes $[z]_i^A = [x]_i^A - \frac{1}{2}[\delta]_i^A$.

TABLE III
TRUTH TABLE

u	v	ω	$s (x > 0)$
0	0	1	1
0	0	0	0
0	1	1	0
0	1	0	1
1	0	1	0
1	0	0	1
1	1	1	1
1	1	0	0

3) *Secure Comparison (SC)*: For the x , the functionality computes the sign. If $x > 0$, $s = 1$, else $s = 0$. The SC we proposed is the most crucial building blocks to our framework, which is applied for ReLU and max pooling. In edge computing scenario, we need to provide a low-latency inference service for the resource-constrained edge cloud servers. Therefore, SC has the characteristics of low rounds and low communication in order to better apply in edge computing. To perform SC once, P_0, P_1 only need to interact once and the communication overhead only requires $3l$.

Specifically, P_1 holds $S_{1,0}$ and $P_{1,1}$. P_0 holds a choice bit $u \in \{0, 1\}$. SC returns $[s]_0^B = u \oplus \omega$ to P_0 and $[s]_1^B = v$ to P_1 , where $u \oplus v \oplus \omega = s$. To verify the correctness, we give a truth table in Table III. In Table III, we enumerate all possible results. A pseudorandom function (PRF) F is applied to generate some random integers between P_0 and P_1 . We assume that the PRF key of P_0 and P_1 is $k_{\{0,1\}}$, which has been generated and sent to P_i by PS. The functionality $\mathcal{F}_{SC}([x]^A)$ is described as follows.

- 1) P_i generates two random integers $R_1 = F_{k_{\{0,1\}}}(1)$, $R_3 = F_{k_{\{0,1\}}}(3)$ ($R_1, R_3 \in \mathbb{Z}_{2^{l/4}}, R_1 > R_3$).
- 2) P_0 chooses $u \leftarrow \{0, 1\}$ randomly and generates $R_2 \in \mathbb{Z}_{2^{l/4}}$ ($R_1 > R_2 + R_3$) randomly. Then, P_0 computes $S_0 = R_2 + (-1)^u \cdot R_1 \cdot [x]_0^A$ and sends S_0 to P_1 .
- 3) P_1 chooses $v \leftarrow \{0, 1\}$ randomly and generates $R_4, R_5 \in \mathbb{Z}_{2^{l/4}}$ ($R_4 < R_3, R_5 < R_3$) randomly. Then, P_1 computes $S_{1,0}, S_{1,1}$ and sends them to P_0 .

$$\begin{aligned}
S_{1,0} &= (-1)^v \cdot (R_4 + S_0 + R_1 \cdot [x]_1^A) \\
&= (-1)^v \cdot (R_4 + R_2 + (-1)^u \cdot R_1 \cdot [x]_0^A + R_1 \cdot [x]_1^A) \\
S_{1,1} &= (-1)^v \cdot (R_5 + S_0 - R_1 \cdot [x]_1^A) \\
&= (-1)^v \cdot (R_5 + R_2 + (-1)^u \cdot R_1 \cdot [x]_0^A - R_1 \cdot [x]_1^A)
\end{aligned}$$

- 4) P_0 sets $S = S_{1,u}$ and judge the value of ω . Let $\omega = 1$ if $S < 2^{l-1}$, else $\omega = 0$.

Correctness: To prove the correctness of the functionality, we assume that $u = 0, v = 0, x > 0$. In (2), we can obtain that $S_0 = R_2 + R_1 \cdot [x]_0^A$. In (3), $S_{1,0} = R_4 + R_2 + R_1 \cdot x$ and $S_{1,1} = R_5 + R_2 + R_1 \cdot [x]_0^A - R_1 \cdot [x]_1^A$. In (4), we can judge that $S_{1,0} < 2^{l-1}$ and $\omega = 1$. Based on the Truth Table, the sign of x is $s = u \oplus v \oplus \omega = 1$.

B. Protocols

1) *Secure Linear Layer*: In the CNN model, convolutional layers and fully-connected layers belong to linear layers.

Functionality $\mathcal{F}_{SC}([x]^A)$

Input: P_i inputs $[x]_i^A$.

Output: P_i obtains $[s]_i^B$.

1. P_i generates two random integers $R_1 = F_{k_{\{0,1\}}}(1)$, $R_3 = F_{k_{\{0,1\}}}(3)$, $R_1, R_3 \in \mathbb{Z}_{2^{l/4}}, R_1 > R_3$. P_0 chooses $u \leftarrow \{0, 1\}$.
2. P_0 generates $R_2 \in \mathbb{Z}_{2^{l/4}}$ and satisfying $R_1 > R_2 + R_3$.
3. P_0 computes $S_0 = R_2 + (-1)^u \cdot R_1 \cdot [x]_0^A$ and sends to P_1 .
4. P_1 chooses $v \leftarrow \{0, 1\}$ and sets $[s]_1^B = v$.
5. P_1 generates two random integers $R_4, R_5 \in \mathbb{Z}_{2^{l/4}}$ and satisfying $R_4 < R_3, R_5 < R_3$.
6. P_1 computes $S_{1,0} = (-1)^v \cdot (R_4 + S_0 + R_1 \cdot [x]_1^A)$ and $S_{1,1} = (-1)^v \cdot (R_5 + S_0 - R_1 \cdot [x]_1^A)$.
7. P_1 sends $S_{1,0}, S_{1,1}$ to P_0 .
8. P_0 sets $S = S_{1,u}$ and makes a judgment as follows.
if $S < 2^{l-1}$:
 $\omega = 1$
else:
 $\omega = 0$
9. P_0 sets $[s]_0^B = u \oplus \omega$.

Secure Convolutional Layer (SCL): For a convolutional layer, the parameters $\mathbf{W} \in R^{U \cdot V \cdot D \cdot P}$ of the convolutional filter represents a four-dimensional tensor. Each slice matrix $\mathbf{W}^{d,p} \in R^{U \cdot V}$ ($d \in [1, D], p \in [1, P]$) is expressed as a two-dimensional convolutional filter. The feature image $\mathbf{X} \in R^{M \cdot N \cdot D}$ is a three-dimensional tensor. Each slice matrix $\mathbf{X}^d \in R^{M \cdot N}$ ($d \in [1, D]$) is expressed as the input feature. In the process of convolution computation, $\mathbf{X}^1, \mathbf{X}^2, \dots, \mathbf{X}^D$ is convoluted with $\mathbf{W}^{p,1}, \mathbf{W}^{p,2}, \dots, \mathbf{W}^{p,D}$ ($\mathbf{Z}^p = \mathbf{W}^p \cdot \mathbf{X}$). Then, adding the convolution results and outputting $\mathbf{Z} \in R^{M' \cdot N' \cdot P}$. The normalization parameters are (α_k, β_k) and the normalization operation is $\mathbf{Z} \cdot \alpha_k + \beta_k$ in [5].

Through the above analysis, we observe that the convolution computation is combined with multiplication and addition. For addition secret sharing, the addition operation can be computed locally and the multiplication will be completed by interactive computation. Due to contain many multiplication operations in a convolutional layer, the communication overhead is large [4], [5]. In [5], convolution computation and normalization operations are divided into two steps, which significantly increases the communication overhead. In [4], the normalization operations are not be considered. Therefore, we propose a secure and efficient convolution computation protocol for the edge computing scenario, which completes both convolution computation and normalization operations at the same time. The computation process is described in Protocol Π_{SCL} .

Secure Fully-connected Layer (SFL): The computation method of secure fully-connected layer Π_{SFL} is the same as secure convolutional layer. For the parameters $\mathbf{W} \in R^{U \cdot V \cdot D \cdot P}$ and the input values $\mathbf{X} \in R^{M \cdot N \cdot D}$, $U = M$ and $V = N$. After the secure fully-connected layer computation, we assume that the output values are $[\mathbf{Z}]_i^A$.

2) *Secure ReLU (SR)*: Based on the CNN model, the outputs of Π_{SCL} are the inputs of ReLU. To design a secure ReLU computation protocol, we transform the ReLU function $ReLU(x) =$

Protocol Π_{SCL}

Input: P_i inputs $\langle W^p \rangle_i = (\langle w_1^p \rangle_i, \langle w_2^p \rangle_i, \dots, \langle w_m^p \rangle_i)$,
 $\langle \alpha_k \rangle_i = (\langle \alpha_1^k \rangle_i, \langle \alpha_2^k \rangle_i, \dots, \langle \alpha_m^k \rangle_i)$, $[\beta_k]_i^A$,
 $\langle X^j \rangle_i = (\langle x_1^j \rangle_i, \langle x_2^j \rangle_i, \dots, \langle x_m^j \rangle_i)$,
 $p \in [1, P], j \in [1, t], m = U \cdot V \cdot D$.
Output: P_i obtains the result shares
 $[Z^p]_i^A = ([z_1^p]_i^A, [z_2^p]_i^A, \dots, [z_t^p]_i^A)$.
Offline:
1. PS generates $[\delta_{w,x}^{p,j}]_i^A, [\delta_{x,\alpha_k}^j]_i^A, [\delta_{w,\alpha_k}^p]_i^A, [\delta_{w,x,\alpha_k}^{p,j}]_i^A$.
 $[\delta_{w,x}^{p,j}]_i^A = ([\delta_1^p \cdot \delta_1^j]_i^A, [\delta_2^p \cdot \delta_2^j]_i^A, \dots, [\delta_m^p \cdot \delta_m^j]_i^A)$,
 $[\delta_{x,\alpha_k}^j]_i^A = ([\delta_1^j \cdot \delta_1]_i^A, [\delta_2^j \cdot \delta_2]_i^A, \dots, [\delta_m^j \cdot \delta_m]_i^A)$,
 $[\delta_{w,\alpha_k}^p]_i^A = ([\delta_1^p \cdot \delta_1]_i^A, [\delta_2^p \cdot \delta_2]_i^A, \dots, [\delta_m^p \cdot \delta_m]_i^A)$,
 $[\delta_{w,x,\alpha_k}^{p,j}]_i^A = ([\delta_1^p \cdot \delta_1^j \cdot \delta_1]_i^A, [\delta_2^p \cdot \delta_2^j \cdot \delta_2]_i^A, \dots, [\delta_m^p \cdot \delta_m^j \cdot \delta_m]_i^A)$.
2. PS sends them to P_i by a secure channel.
Online:
for $p = 1 \rightarrow P$:
for $j = 1 \rightarrow t$:
for $q = 1 \rightarrow m$:
1. P_i computes
 $[U_z^{p,j,q}]_i^A = \mathcal{F}_{3-mul}(\langle w_q^p \rangle_i, \langle x_q^j \rangle_i, \langle \alpha_q^k \rangle_i)$.
end for
2. $[U_z^j]_i^A = \sum_{q=1}^m [U_z^{p,j,q}]_i^A + [\beta_k]_i^A$.
end for
3. $[U_z^p]_i^A = ([U_z^1]_i^A, [U_z^2]_i^A, \dots, [U_z^t]_i^A)$.
4. $[Z^p]_i^A = T(T([U_z^p]_i^A))$.
end for

Protocol Π_{ReLU}

Input: P_i inputs $[x]_i^A$.
Output: P_i obtains $[y]_i^A$.
1. P_i invokes $[s]_i^B = \mathcal{F}_{SC}([x]_i^A)$.
2. P_i invokes $[y]_i^A = \mathcal{F}_S([s]_i^B, [x]_i^A, [r]_i^A)$.

$\max(x, 0)$ into $y = (x > 0 ? 1 : 0) \cdot x$, which consists of a $\mathcal{F}_{SC}$ and a $\mathcal{F}_S$. The secure ReLU computation is described in Protocol Π_{ReLU} . P_0 and P_1 perform the protocol as follows. As the most complex operation in the inference process of CNN model, the protocol Π_{ReLU} is lightweight and efficient, which is suitable for the edge computing scenario.

- 1) P_i holds $[x]_i^A$.
- 2) P_i invokes $[s]_i^B = \mathcal{F}_{SC}([x]_i^A)$.
- 3) We cite the method proposed in [4] to generate the secret sharing of 0 in the offline phase, satisfying $0 = [r]_0^A + [r]_1^A$. Then, P_i invokes $[y]_i^A = \mathcal{F}_S([s]_i^B, [x]_i^A, [r]_i^A)$.
- 3) **Secure Pooling Layer:** We assume that the inputs of pooling layer is $X^d \in R^{M \cdot N}$, $d \in [1, D]$ and divide X^d into different vectors $R_{m,n}^d$ ($m \in [1, M'], n \in [1, N'], t = |R_{m,n}^d|$). Then, we will down sampling for each vector and obtain a value to express the vector.

Protocol Π_{SMP}

Input: For each vector $R_{m,n}^d$, P_i inputs
 $([x_1]_i^A, [x_2]_i^A, \dots, [x_t]_i^A)$
Output: P_i obtains $[y_{m,n}^d]_i^A$.
Online:
1. P_i sets $[y_{m,n}^d]_i^A = [x_1]_i^A$
for $j = 2 \rightarrow t$:
2. P_i computes $[y_{m,n}^d - x_j]_i^A = [y_{m,n}^d]_i^A - [x_j]_i^A$.
3. P_i invokes $[s]_i^B = \mathcal{F}_{SC}([y_{m,n}^d - x_j]_i^A)$.
4. P_i invokes $[z]_i^A = \mathcal{F}_S([s]_i^B, [y_{m,n}^d - x_j]_i^A, [r]_i^A)$.
5. P_i computes $[y_{m,n}^d]_i^A = [z]_i^A + [x_j]_i^A$
end for

The average pooling is expressed as:

$$y_{m,n}^d = \frac{1}{t} \sum_{i \in L_{m,n}^d} x_i$$

The max pooling is expressed as:

$$y_{m,n}^d = \max_{i \in t} x_i$$

Secure Average Pooling (SAP): The secure average pooling Π_{SAP} is completed by P_i locally. For each vector $R_{m,n}^d$, P_i holds the shares $([x_1]_i^A, [x_2]_i^A, \dots, [x_t]_i^A)$. P_i will compute as follows.

$$[y_{m,n}^d]_i^A = \frac{\sum_{j=1}^t [x_j]_i^A}{t}$$

Secure Max Pooling (SMP): The secure max pooling is completed by P_i with interactions. For each vector $R_{m,n}^d$, P_i holds $([x_1]_i^A, [x_2]_i^A, \dots, [x_t]_i^A)$. To compute the maximum value, P_i sets $[y_{m,n}^d]_i^A = [x_1]_i^A$ as the current maximum value. Then, P_i compares the current maximum value with the rest $t - 1$ values and updates the current maximum value. The computation process is shown in Protocol Π_{SMP} .

To reduce the number of interactions of P_0 and P_1 , we can invoke $\mathcal{F}_{SC}$ and $\mathcal{F}_S$ in parallel. If $t = 4$, $\{\mathcal{F}_{SC}([x_1 - x_2]_i^A), \mathcal{F}_S([s]_{1,2}^B, [x_1 - x_2]_i^A, [r]_i^A)\}$ and $\{\mathcal{F}_{SC}([x_3 - x_4]_i^A), \mathcal{F}_S([s]_{3,4}^B, [x_3 - x_4]_i^A, [r]_i^A)\}$ can be executed in parallel.

4) **Conversion:** Based on the CNN model, P_i performs SCL or SFL after the secure pooling layer computation (or secure ReLU computation). For the secure pooling layer computation (or secure ReLU computation), the semantic of secret sharing is $[\cdot]^A$. For the SCL or SFL, the inputs semantic of secret sharing is $\langle \cdot \rangle$. Therefore, P_i needs to convert $[x]_i^A$ to $\langle x \rangle_i$. The conversion process is described in Protocol $\Pi_{[\cdot]^A \rightarrow \langle \cdot \rangle}$.

- 1) P_i holds $[x]_i^A$.
- 2) P_1 computes $[U]_1^A = [x]_1^A + [\delta]_1^A$ and sends to P_0 . It should be noted that $[\delta]_i^A$ has been generated by PS in the offline phase for the secure convolutional layer computation.
- 3) P_0 computes $[U]_0^A = [x]_0^A + [\delta]_0^A$, $U = [U]_0^A + [U]_1^A$ and sends U to P_1 .
- 4) P_i completes the conversion, $\langle x \rangle_i = (U, [\delta]_i^A)$.

Protocol $\Pi_{[\cdot]^A \rightarrow \langle \cdot \rangle}$ **Input:** P_i inputs $[x]_i^A$.**Output:** P_i obtains $\langle x \rangle_i = (U, [\delta]_i^A)$.

Offline:

1. PS generates $[\delta]_i^A$.

Online:

1. P_1 computes $[U]_1^A = [x]_1^A + [\delta]_1^A$ and sends to P_0 .2. P_0 computes $[U]_0^A = [x]_0^A + [\delta]_0^A$, $U = [U]_0^A + [U]_1^A$ and sends U to P_1 .3. P_i obtains $\langle x \rangle_i = (U, [\delta]_i^A)$.

Optimized communication: For a complex CNN model, we assume that one segment structure of the model as follows.

$$\dots \text{Convolution} \rightarrow \text{ReLU} \rightarrow \text{Max Pooling} \dots$$

Then, we construct a secure inference process based on our proposed protocols. The two semantics $[\cdot]^A$ and $\langle \cdot \rangle$ are applied in different layers. It should be noted that $\langle \cdot \rangle$ brings lower communication overhead than $[\cdot]^A$ when making the convolution operations. For Π_{ReLU} and Π_{SMP} , $[\cdot]^A$ and $\langle \cdot \rangle$ bring the same communication overhead. But if all protocols are designed based on $\langle \cdot \rangle$, $\Pi_{[\cdot]^A \rightarrow \langle \cdot \rangle}$ will be invoked many times in such a segment structure. To reduce the communication overhead, we design protocols by combining $[\cdot]^A$ and $\langle \cdot \rangle$. Therefore, $\Pi_{[\cdot]^A \rightarrow \langle \cdot \rangle}$ is only invoked once in such a segment structure. The computation process is as follows.

$$\langle \cdot \rangle \xrightarrow{\Pi_{SCL}} [\cdot]^A \xrightarrow{\Pi_{ReLU}} [\cdot]^A \xrightarrow{\Pi_{SMP}} [\cdot]^A \xrightarrow{\Pi_{[\cdot]^A \rightarrow \langle \cdot \rangle}} \langle \cdot \rangle$$

VI. PROPOSED FRAMEWORK

In this section, we completely describe the process of our proposed secure inference framework.

A. Ciphertext Setup and Truncation

The floating-point value x will be converted into integer x' in Z_{2^l} before sharing, where the $[0, 2^{l-1} - 1]$ denote the non-negative integers and $[2^{l-1}, 2^l - 1]$ denote the negative integers. For $x' = x \cdot 2^{l_q}$, the l_q denotes the fractional part. After each multiplication operation, the fractional part of result will become $2 \cdot l_q$. To maintain the same bits of fractional part, the truncation operation will be performed. Our framework adopts the truncation method in [34].

B. Inference Service Outsourcing

In this phase, SP outsources the inference service to CS_0 and CS_1 . For the CNN model of SP, we assume that the number of linear layers is L . The parameters of CNN model are expressed as $\mathbf{W}_1, \mathbf{W}_2, \dots, \mathbf{W}_L$, $\mathbf{W}_i \in R^{U \cdot V \cdot D \cdot P}$ and the normalization parameters are expressed as (α_i, β_i) , $i \in [1, L]$.

To outsource the CNN model $\mathbf{W}_k, (\alpha_k, \beta_k)$, $k \in [1, L]$, we express the $\mathbf{W}_k$ as $\mathbf{W}^1, \mathbf{W}^2, \dots, \mathbf{W}^P$ and $\mathbf{W}^p = (w_1^p, w_2^p, \dots, w_m^p)$ ($m = U \cdot V \cdot D$, $p \in [1, P]$). For $\mathbf{W}^p$, PS generates m random integers for each parameter w_i^p , which is

$\delta_w^p = (\delta_1^p, \delta_2^p, \dots, \delta_m^p)$, $\delta_i^p \in Z_{2^l}$ and sends to SP secretly. For α_k , PS generates m random integers $\delta_{\alpha_k} = (\delta_1, \delta_2, \dots, \delta_m)$ and sends to SP secretly. For w_i^p, α_k ($i \in [1, m]$), SP computes $U_i^p = w_i^p + \delta_i^p$, $U_i^p = \alpha_k + \delta_i$ and obtains $\langle w_i^p \rangle_j = (U_i^p, [\delta_i^p]_j^A)$, $\langle \alpha_i^k \rangle_j = (U_i^p, [\delta_i]_j^A)$, $j \in \{0, 1\}$. For β_k , SP generates a random integer $r \in Z_{2^l}$ and sets $[\beta_k]_0^A = \beta_k - r$, $[\beta_k]_1^A = r$. Then, SP sends $\langle w_i^p \rangle_j, \langle \alpha_i^k \rangle_j, [\beta_k]_j^A$ to CS_j secretly.

C. Inference Request Generating

PS generates random integers $\delta_x^j = (\delta_1^j, \delta_2^j, \dots, \delta_m^j)$, $\delta_i^j \in Z_{2^l}$, $j \in [1, t]$ and sends to BD secretly. BD holds the data $\mathbf{X}$. To input $\mathbf{X}$ into the CNN model, BD transforms $\mathbf{X}$ into ciphertext $\langle \mathbf{X}^j \rangle_i = (U_x^j, [\delta_x^j]_i^A)$, $j \in [1, t]$. Then, BD sends $\langle \mathbf{X}^j \rangle_i$ to CS_i .

D. Secure CNN Inference

A typical convolutional network is composed of linear layer, ReLU and pooling layer. We choose different building blocks for different layers. Therefore, our proposed protocol can be extended to any CNN model. After completing the last fully-connected layer computation, CS_i sends $[\mathbf{Z}]_i^A$ to BD secretly. The inference results $y = [\mathbf{Z}]_0^A + [\mathbf{Z}]_1^A$ are computed by BD.

VII. SECURITY ANALYSIS

In this section, we prove that our proposed framework can guarantee security in the semi-honest adversaries. We make a simulation-based security proof based on the universal composability framework [35], [36]. The simulators are constructed to simulate the view of semi-honest adversaries. In our proposed framework, all data will be encrypted with secret sharing.

The BD needs to generate the inference request and obtain the inference results. The SP only needs to outsource the CNN model. It can be seen that BD and SP will not participant in the inference process. Therefore, we only analyze the inference process completed by CS_0 and CS_1 . The adversaries are expressed as $\mathcal{A}$. Sim_i denotes the simulator of CS_i . We use the lemmas proposed in [4], [28], [29].

Lemma 1: If the sub protocols are secure, then the protocol built on these sub protocols is also secure.

Lemma 2: If a random integer r is uniformly distributed on Z_{2^l} and independent from $x \in Z_{2^l}$, then $x \pm r$ ($x \cdot r$) is also uniformly random and independent from x .

Lemma 3: $\mathcal{F}_{3-mul}$ and $\mathcal{F}_S$ are secure [4], [29].

Security Definition: We assume that Π is a two-party protocol for computing a function f . Let $view_i^\Pi$ denote the view of CS_i in the real world. In the semi-honest adversary model, the protocol is secure such that

$$Sim_i(x_i, f) \stackrel{c}{=} view_i^\Pi$$

where $\stackrel{c}{=}$ denotes computational indistinguishability.

Theorem 1: The $\mathcal{F}_{SC}$ is secure against honest-but-curious adversaries $\mathcal{A}$.

Proof 1: The secure comparison computation is completed by CS_0 and CS_1 with interactions. The view of CS_0 is

$$view_0^{\Pi_{\mathcal{F}_{SC}}}([x]_0^A, R_1, R_3, u, R_2, S_0, S_{1,0}, S_{1,1}, \omega, [s]_0^B)$$

and the view of CS_1 is

$$view_1^{\Pi_{\mathcal{F}_{SC}}}([x]_1^A, R_1, R_3, v, R_4, R_5, S_{1,0}, S_{1,1}, S_0, [s]_1^B)$$

The process by which simulators construct the view of adversaries is as follows:

1) In the real world, CS_0 receives $S_{1,0}, S_{1,1}$. In the ideal world, Sim_0 generates random integers $S'_{1,0}, S'_{1,1}$. Due to $S_{1,0}, S_{1,1}$ and $S'_{1,0}, S'_{1,1}$ are both uniform random integers and uncorrelated, the views of the adversary in the real world and the ideal world are computationally indistinguishable.

$$Sim_0([x]_0^A, R'_1, R'_3, u', R'_2, S'_0, S'_{1,0}, S'_{1,1}, \omega', [s]_0^B) \stackrel{c}{=} view_0^{\Pi_{\mathcal{F}_{SC}}}([x]_0^A, R_1, R_3, u, R_2, S_0, S_{1,0}, S_{1,1}, \omega, [s]_0^B)$$

2) In the real world, CS_1 receives S_0 . In the ideal world, Sim_1 generates a random integer S'_0 . Due to S_0 and S'_0 are both uniform random integers and uncorrelated, the views of the adversary in the real world and the ideal world are computationally indistinguishable.

$$Sim_1([x]_1^A, R'_1, R'_3, v', R'_4, R'_5, S'_{1,0}, S'_{1,1}, S'_0, [s]_1^B) \stackrel{c}{=} view_1^{\Pi_{\mathcal{F}_{SC}}}([x]_1^A, R_1, R_3, v, R_4, R_5, S_{1,0}, S_{1,1}, S_0, [s]_1^B)$$

Therefore, the $\mathcal{F}_{SC}$ is secure.

Theorem 2: The Π_{SCL} is secure against honest-but-curious adversaries $\mathcal{A}$.

Proof 2: In the online phase, the computation process is completed by CS_i locally. The view of CS_i is expressed as

$$(\langle W^p \rangle_i, \langle \alpha_k \rangle_i, [\beta_k]_i^A, \langle X^j \rangle_i, [\delta_{w,x}^{p,j}]_i^A, [\delta_{w,\alpha_k}^{p,j}]_i^A, [\delta_{x,\alpha_k}^j]_i^A, [\delta_{w,x,\alpha_k}^{p,j}]_i^A)$$

We can find that CS_i completes the computation process by using random integers and secret shares in the real world. At the same time, Sim_i can construct random integers and secret shares that are indistinguishable from the adversary. Therefore, the views of the adversary in the real world and the ideal world are computationally indistinguishable.

$$Sim_i(\langle W^p \rangle_i, \langle \alpha'_k \rangle_i, [\beta'_k]_i^A, \langle X'^j \rangle_i, [\delta_{w,x}^{p,j}]_i^A, [\delta_{w,\alpha_k}^{p,j}]_i^A, [\delta_{x,\alpha_k}^j]_i^A, [\delta_{w,x,\alpha_k}^{p,j}]_i^A) \stackrel{c}{=} view_i^{\Pi_{SCL}}(\langle W^p \rangle_i, \langle \alpha_k \rangle_i, [\beta_k]_i^A, \langle X^j \rangle_i, [\delta_{w,x}^{p,j}]_i^A, [\delta_{w,\alpha_k}^{p,j}]_i^A, [\delta_{x,\alpha_k}^j]_i^A, [\delta_{w,x,\alpha_k}^{p,j}]_i^A)$$

The Π_{SCL} is secure.

Theorem 3: The Π_{SFL} is secure against honest-but-curious adversaries $\mathcal{A}$.

Proof 3: The computation process of Π_{SFL} is the same as Π_{SCL} . Therefore, the Π_{SFL} is also secure.

Theorem 4: The Π_{ReLU} is secure against honest-but-curious adversaries $\mathcal{A}$.

Proof 4: The Π_{ReLU} is completed by CS_0 and CS_1 . In the real world, CS_i invokes $\mathcal{F}_{SC}$ and $\mathcal{F}_S$. The security of $\mathcal{F}_{SC}$ and $\mathcal{F}_S$ has been proved. Therefore, Π_{ReLU} is also secure based on Lemma 1.

Theorem 5: The secure pooling layer is secure against honest-but-curious adversaries $\mathcal{A}$.

Proof 5: For the SAP, CS_i completes the computation process by using secret shares locally in the real world. Therefore, Sim_i can construct secret shares that are indistinguishable from the adversary. SAP is secure.

For the SMP, CS_i invokes $\mathcal{F}_{SC}$ and $\mathcal{F}_S$. The security of $\mathcal{F}_{SC}$ and $\mathcal{F}_S$ has been proved. Therefore, SMP is also secure based on Lemma 1.

Theorem 6: The conversion $\Pi_{[A] \rightarrow \langle \cdot \rangle}$ is secure against honest-but-curious adversaries $\mathcal{A}$.

Proof 6: In the online phase, the process by which simulators construct the view of adversaries in the ideal world is as follows:

- 1) When performing the protocol in the real world, CS_0 receives $[U]_1^A$. In the ideal world, Sim_0 generates a random integer $[U']_1^A$. Due to $[U]_1^A$ and $[U']_1^A$ are both uniform random integers and uncorrelated, the views of the adversary in the real world and the ideal world are computationally indistinguishable.
- 2) When performing the protocol in the real world, CS_1 receives U . In the ideal world, Sim_1 generates a random integer U' . Due to U and U' are both uniform random integers and uncorrelated, the views of the adversary in the real world and the ideal world are computationally indistinguishable.

$$Sim_i([x]_i^A, [\delta]_i^A, [U']_i^A, U') \stackrel{c}{=} view_i^{\Pi_{[A] \rightarrow \langle \cdot \rangle}}([x]_i^A, [\delta]_i^A, [U]_i^A, U)$$

From the above analysis, the conversion is secure.

Theorem 7: The secure CNN inference is secure against honest-but-curious adversaries $\mathcal{A}$.

Proof 7: The secure CNN inference consists of the proposed building blocks. At the same time, we have proved that all building blocks are secure. In the secure CNN inference process, each building block is performed serially. Based on Lemma 1 and Theorems 1–6, the secure CNN inference is secure.

VIII. PERFORMANCE EVALUATION

In this section, we implement our proposed framework and evaluate the performance.

In our experiments, We first train two CNN models using Pytorch on two standard datasets MNIST [37] and CIFAR-10 [38], respectively. The two trained models are named Model-1 and Model-2. Then, the inference performance evaluation is conducted on these two models. The structure of the CNN models and the ciphertext inference process are shown in Fig. 3

For the experimental setup, we evaluate the performance in the two network settings. In the LAN network setting, we simulate CS_i with a laptop, which is equipped with an Intel(R) Core(TM)i7-10510 U CPU operating at 1.80 GHz with 1.9 G in Ubuntu 22.04.3. In the WAN network setting, we simulate CS_0

TABLE IV
COMPARISON OVERVIEW

Schemes	Performance		Function		Security	
	Low rounds	Low communication	Image inference	Edge computing friendly	Model privacy	Results privacy
[5]	×	×	✓	×	✓	✓
[4]	×	×	✓	×	✓	✓
[28]	×	✓	✓	×	×	✓
[32]	×	×	✓	×	✓	✓
Ours	✓	✓	✓	✓	✓	✓

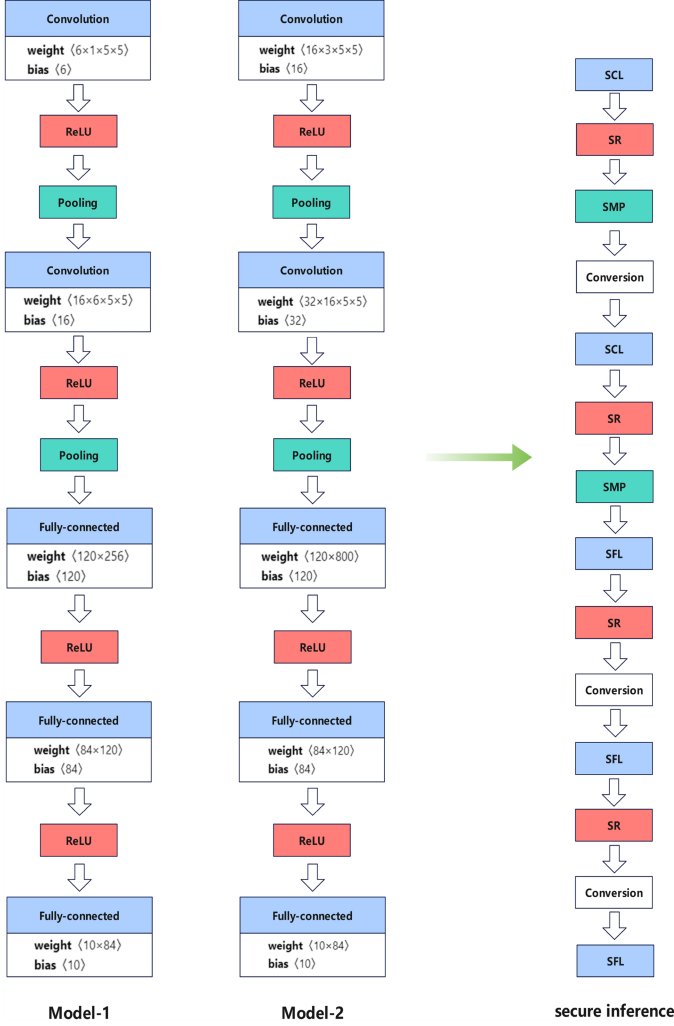


Fig. 3. Model structure and ciphertext inference process.

with the laptop used in the LAN setting and simulate CS_1 with a remote server. The remote server is equipped with an Intel(R) Xeon(R) Silver 4208 CPU operating at 2.10 GHz. We implement our proposed framework and the compared schemes [4], [5] with Python and achieve two-party communication based on the socket package.

A. Performance of Our Proposed Framework

The secure CNN inference consists of the proposed secure building blocks. We test the computation time and communication overhead of the proposed building blocks in LAN setting.

TABLE V
THE COMPUTATION OVERHEAD OF THE PROPOSED FRAMEWORK

The ring size (Z_{2^l})	Π_{SCL} (s)	Π_{ReLU} (s)	Π_{SMP} (s)
64	0.36	0.17	0.28
128	0.44	0.27	0.31
256	0.67	0.34	0.32

TABLE VI
THE COMMUNICATION OVERHEAD OF THE PROPOSED BUILDING BLOCKS

The ring size (Z_{2^l})	Π_{SCL} (KB)	Π_{ReLU} (KB)	Π_{SMP} (KB)
64	0	206.45	206.39
128	0	401.75	401.76
256	0	790.02	790.03

The evaluated building blocks include **SCL**, **SR** and **SMP**. These building blocks are the main steps for the secure CNN inference. The evaluated convolutional layer, ReLU and max pooling correspond to the first three layers of Model-1. Based on the security consideration and performance evaluation, we set the ring size as Z_{2^l} ($l = 64, 128, 256$).

The evaluation results are shown in Table V and Table VI. Through the results, the computation time increases with the increase of l . In our proposed framework, there is no communication overhead for **SCL**. For **SR** and **SMP**, the communication overhead increases with the increase of l . In order to balance security and efficiency, we set $l = 64$ in the subsequent evaluation.

B. Comparison Analysis

In comparison analysis, we make a comparison with [4], [5], [28], [32]. The comparison overview is shown in Table IV. In the scheme [28], the authors assume that the CNN model is public, which can not provide model privacy. In practical scenarios, this assumption lowers the security level. In our proposed scheme and [4], [5], [32], the CNN model is privacy property of a company and needs to be provided privacy-preserving during the inference process. In the scheme [4], [5], their schemes require many interactions and much communication overhead when performing secure ReLU and secure max pooling. For the scheme [32], the characteristics of their proposed secure linear layer protocol are high rounds and communication. Lower rounds and communication are crucial for edge cloud servers to complete a real-time secure CNN inference in the edge computing scenario. Therefore, our proposed framework has more advantages than other schemes.

1) *Theoretical analysis:* In the process of **secure convolutional layer**, our proposed protocol compute the convolution

TABLE VII
COMPLEXITY ANALYSIS FOR SECURE CONVOLUTIONAL LAYER

Schemes	Computation	Communication	Rounds
[5], Convolution and Normalization	$O(2Pt)$	$4(UVDtP + M'N'P) \cdot l$	1
[4], Convolution (No normalization)	$O(Pt)$	$4UVDtP \cdot l$	1
[28], Convolution (No normalization)	$O(Pt)$	0	0
Ours , Convolution and Normalization	$O(Pt)$	0	0

TABLE VIII
COMPLEXITY ANALYSIS FOR SECURE ReLU

Schemes	Computation	Communication	Rounds
[5]	$O(2\log_2 l)$	$8l + 4\log_2 l$	$2 + \log_2 l$
[4]	$O(1)$	$13l$	4
[28]	$O(l)$	$10l - 4$	l
[32]	$O(l)$	$\lambda \cdot l + 18l$	$\log_2 l + 2$
Ours	$O(1)$	$7l$	2

TABLE IX
COMPLEXITY ANALYSIS FOR SECURE MAX POOLING

Schemes	Computation	Communication	Rounds
[5]	$O(2\log_2 l \cdot t)$	$(8l + 4\log_2 l) \cdot t$	$(2 + \log_2 l) \cdot t$
[4]	$O(t)$	$13l \cdot t$	$4\log_2 t$
[28]	$O(lt)$	$(10l - 4)t$	$l \cdot \log_2 t$
[32]	$O(lt)$	$(t - 1) \cdot (\lambda \cdot l + 14l)$	$\log_2 l \cdot \log_2 t$
Ours	$O(t)$	$7l \cdot t$	$2\log_2 t$

values and provide normalization operations. The results are shown in Table VII. The computation complexity of our proposed **secure convolutional layer** is $O(Pt)$. To reduce the additional communication overhead and optimize the computation process, the semantic of the input values is $\langle \cdot \rangle$ and the semantic of the output values is $[\cdot]^A$. The communication complexity is 0 and the rounds are 0. In [4], [5], their schemes need more communication overhead and rounds to complete the convolution computation. Although the communication overhead and rounds of [28] are 0, their scheme does not support the privacy of the CNN model. For the normalization operation, the scheme [5] needs more computation overhead and there is no normalization operation in [4], [28].

The Tables VIII and IX show the results of theoretical analysis. For our proposed **secure ReLU**, the computation complexity is $O(1)$ and the communication complexity is $7l$. The computation overhead is significantly lower than [5], [28], [32]. The communication overhead is significantly lower than [4], [5], [28], [32]. The rounds of performing a protocol are also a main factor that affecting the efficiency. It should be noted that the rounds of our proposed protocol and [4] are a constant. In [5], [28], [32], the rounds is related to the size of l . The λ is the security parameter and $\lambda = 128$. For the **secure max pooling**, the number of operation is related to the size of t . Based on the theoretical analysis, we can find that the proposed **secure ReLU** and **secure max pooling** are more efficient.

2) *Performance analysis*: From Table IV, the scheme [28] does not provide the model privacy. In order to ensure fairness in comparison analysis, we no longer compare scheme [28] during performance analysis. At the same time, the scheme [32]

TABLE X
COMMUNICATION OVERHEAD: CONVOLUTION, ReLU, MAX POOLING

Schemes	Convolution (KB)	ReLU (KB)	Max Pooling (KB)
[5]	3579.98	2826.94	2826.79
[4]	3442.35	550.71	550.89
Ours	0	206.45	206.39

TABLE XI
COMPUTATION TIME: CONVOLUTION, ReLU AND MAX POOLING

Schemes	Convolution (s)		ReLU (s)		Max Pooling (s)	
	LAN	WAN	LAN	WAN	LAN	WAN
[5]	0.86	11.31	3.96	13.28	3.13	15.59
[4]	0.48	8.99	0.21	2.43	0.29	2.32
Ours	0.36	0.31	0.17	1.31	0.28	1.29

TABLE XII
THE COMPUTATION TIME OF SECURE CNN INFERENCE

Schemes	Model-1 (s)		Model-2 (s)	
	LAN	WAN	LAN	WAN
[5]	11.03	76.13	62.53	393.82
[4]	2.65	35.26	34.01	284.27
Ours	2.23	10.41	15.16	50.87

achieves the linear layer computation based on the oblivious transfer and HE, which brings much more computation and communication costs than the schemes using arithmetic secret sharing technology [4]. Therefore, we implement the schemes [4], [5] and make comparison with our proposed scheme.

Then, we test the computation and communication costs of the main building blocks (Convolution, ReLU and Pooling). We set $l = 64$. The evaluated convolutional layer, ReLU and max pooling correspond to the first three layers of Model-1. The results of communication overhead are shown in Table X. For the computation time, we evaluate the building blocks in the LAN setting and WAN setting. The results are shown in Table XI. It can be seen that the computation time of our proposed building blocks is less than 1 s in the LAN setting.

From Tables X and XI, the computational and communication costs of our proposed building blocks are all less than the schemes in [4], [5]. For the convolutional layer, our proposed protocol improves the computational efficiency by 25% in the LAN setting and 96.6% in the WAN setting compared with [4]. Compared with [5], we improve the computational efficiency by 58.1% in the LAN setting and 97.3% in the WAN setting. For the ReLU, we improve the computational efficiency by 19% in the LAN setting and 46.1% in the WAN setting compared with [4]. Compared with [5], we improve the computational efficiency by 95.7% in the LAN setting and 90.1% in the WAN setting. For the max pooling, we gain over 91.1% and 91.7% improvements on the computational efficiency compared with [5] in the LAN and WAN settings respectively. Although the computation time in the LAN setting of [4] and our is similar, we improve the computational efficiency by 44.4% in the WAN setting.

To better test the performance of the proposed secure CNN inference, we make a complete evaluation for computation time and communication overhead with Model-1 and Model-2 in the LAN and WAN settings. The results are shown in Table XII and XIII.

TABLE XIII
THE COMMUNICATION OF SECURE CNN INFERENCE

Schemes	Model-1 (MB)	Model-2 (MB)
[5]	18.45	116.48
[4]	12.39	95.51
Ours	0.56	1.93

From Table XII, we can obtain that the computation time of our scheme is shorter. For the Model-1, we improve the computational efficiency by 15.8% in the LAN setting and 70.5% in the WAN setting compared with [4]. Compared with [5], we improve the computational efficiency by over 79.8% in the LAN setting and 86.3% in the WAN setting. For the Model-2, we improve the computational efficiency by over 55.4% in the LAN setting and 82.1% in the WAN setting compared with [4]. Compared with [5], we improve the computational efficiency by over 75.8% in the LAN setting and 87.1% in the WAN setting.

From Table XIII, we can obtain that the communication overhead of our scheme is lower. For the Model-1, we reduce the communication overhead by 95.5% and 96.9% compared with [4] and [5] respectively. For the Model-2, we reduce the communication overhead by 97.9% and 98.3% compared with [4] and [5] respectively. Through the above analysis, our proposed scheme is more efficient.

IX. CONCLUSION

In order to meet the security and efficiency requirements when using CNN model for the inference in the edge computing scenario, we propose a secure and efficient framework to complete the secure CNN inference. Our proposed scheme significantly reduces the number of interactions between CS_0 and CS_1 . By analyzing the structure of CNN model, we highly modularize the CNN inference process. By combining two secret sharing semantics with CNN model structure, we design a more efficient framework. The proposed lightweight secure building blocks greatly improve the computing efficiency of the CS_0 and CS_1 . Based on the comparison analysis, our scheme is more suitable for the edge computing scenario.

REFERENCES

- [1] M.-L. Shi et al., "Effects of dendrobium officinale ultrafine powder on sub-health mice induced by unhealthy lifestyle based on neuroendocrine immune system," *Food Function*, vol. 13, no. 23, pp. 12436–12450, 2022.
- [2] J. Wang, H. Zhu, S.-H. Wang, and Y.-D. Zhang, "A review of deep learning on medical image analysis," *Mobile Netw. Appl.*, vol. 26, no. 1, pp. 351–380, 2021.
- [3] L. Cai, J. Gao, and D. Zhao, "A review of the application of deep learning in medical image classification and segmentation," *Ann. Transl. Med.*, vol. 8, no. 11, 2020.
- [4] J. Wang, D. He, A. Castiglione, B. B. Gupta, M. Karuppiah, and L. Wu, "PCNNCEC: Efficient and privacy-preserving convolutional neural network inference based on cloud-edge-client collaboration," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 5, pp. 2906–2923, Sep./Oct. 2022.
- [5] X. Liu, Y. Zheng, X. Yuan, and X. Yi, "Securely outsourcing neural network inference to the cloud with lightweight techniques," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 1, pp. 620–636, Jan./Feb. 2023.
- [6] T. Liu, X. Xie, and Y. Zhang, "Zkcn: Zero knowledge proofs for convolutional neural network predictions and accuracy," in *Proc. 2021 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2021, pp. 2968–2985.
- [7] Y. Zhang, J. Pan, L. Qi, and Q. He, "Privacy-preserving quality prediction for edge-based iot services," *Future Gener. Comput. Syst.*, vol. 114, pp. 336–348, 2021.
- [8] Q. Feng, D. He, Z. Liu, H. Wang, and K.-K. R. Choo, "SecureNLP: A system for multi-party privacy-preserving natural language processing," *IEEE Trans. Inf. Forensics Secur.*, vol. 15, pp. 3709–3721, 2020.
- [9] Q. Zhang, C. Xin, and H. Wu, "Gala: Greedy computation for linear algebra in privacy-preserved neural networks," 2021, *arXiv:2105.01827*.
- [10] X. Liu, Y. Zheng, X. Yuan, and X. Yi, "Towards secure and lightweight deep learning as a medical diagnostic service," in *Proc. Eur. Symp. Res. Comput. Secur.*, 2021, pp. 519–541.
- [11] S. Wang, C. Ge, L. Zhou, H. Wang, Z. Liu, and J. Wang, "Privacy-preserving classification in multiple clouds healthcare," *IEEE Trans. Serv. Comput.*, vol. 16, no. 1, pp. 493–503, Jan./Feb. 2023.
- [12] J. Liang, Z. Qin, L. Xue, X. Lin, and X. Shen, "Efficient and privacy-preserving decision tree classification for health monitoring systems," *IEEE Internet Things J.*, vol. 8, no. 16, pp. 12528–12539, Aug. 2021.
- [13] B. Pulido-Gaytan et al., "Privacy-preserving neural networks with homomorphic encryption: Challenges and opportunities," *Peer-to-Peer Netw. Appl.*, vol. 14, no. 3, pp. 1666–1691, 2021.
- [14] R. Gilad-Bachrach, N. Dowlan, K. Laine, K. Lauter, M. Naehrig, and J. Wernsing, "Cryptonets: Applying neural networks to encrypted data with high throughput and accuracy," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 201–210.
- [15] X. Liu, B. Wu, X. Yuan, and X. Yi, "Leia: A lightweight cryptographic neural network inference system at the edge," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 237–252, 2022.
- [16] J. Liu, M. Juuti, Y. Lu, and N. Asokan, "Oblivious neural network predictions via minion transformations," in *Proc. 2017 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 619–631.
- [17] M. Li, S. S. Chow, S. Hu, Y. Yan, C. Shen, and Q. Wang, "Optimizing privacy-preserving outsourced convolutional neural network predictions," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 3, pp. 1592–1604, May/Jun. 2022.
- [18] D. Demmler, T. Schneider, and M. Zohner, "Aby-a framework for efficient mixed-protocol secure two-party computation," in *Proc. NDSS*, 2015.
- [19] N. Kumar, M. Rathee, N. Chandran, D. Gupta, A. Rastogi, and R. Sharma, "Cryptflow: Secure tensorflow inference," in *Proc. IEEE 2020 Symp. Secur. Privacy*, 2020, pp. 336–353.
- [20] H. Chaudhari, R. Rachuri, and A. Suresh, "Trident: Efficient 4 PC framework for privacy preserving machine learning," *Netw. Distrib. Syst. Secur. Symp.*, 2020.
- [21] A. Mujtaba, W.-K. Lee, and S. O. Hwang, "Low latency implementations of CNN for resource-constrained IoT devices," *IEEE Trans. Circuits Syst. II: Exp. Briefs*, vol. 69, no. 12, pp. 5124–5128, Dec. 2022.
- [22] B. Xie, T. Xiang, X. Liao, and J. Wu, "Achieving privacy-preserving online diagnosis with outsourced SVM in internet of medical things environment," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 6, pp. 4113–4126, Nov./Dec. 2022.
- [23] X. Li, J. He, P. Vijayakumar, X. Zhang, and V. Chang, "A verifiable privacy-preserving machine learning prediction scheme for edge-enhanced HCPSSs," *IEEE Trans. Ind. Informat.*, vol. 18, no. 8, pp. 5494–5503, Aug. 2022.
- [24] K. Zhao, X. A. Wang, B. Yang, Y. Tian, and J. Zhang, "A privacy preserving homomorphic computing toolkit for predictive computation," *Inf. Process. Manage.*, vol. 59, no. 2, 2022, Art. no. 102880.
- [25] P. Mohassel and P. Rindal, "Aby3: A mixed protocol framework for machine learning," in *Proc. 2018 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 35–52.
- [26] M. Byali, H. Chaudhari, A. Patra, and A. Suresh, "Flash: Fast and robust framework for privacy-preserving machine learning," *PETS*, 2020.
- [27] A. Dalskov, D. Escudero, and M. Keller, "Fantastic four: Honest-majority four-party secure computation with malicious security," in *Proc. 30th USENIX Secur. Symp. (USENIX Secur. 21)*, 2021, pp. 2183–2200.
- [28] K. Huang, X. Liu, S. Fu, D. Guo, and M. Xu, "A lightweight privacy-preserving CNN feature extraction framework for mobile sensing," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 3, pp. 1441–1455, May/Jun. 2021.
- [29] A. Patra, T. Schneider, A. Suresh, and H. Yalame, "ABY2. 0: Improved mixed-protocol secure two-party computation," in *Proc. 30th USENIX Secur. Symp. (USENIX Secur. 21)*, 2021, pp. 2165–2182.
- [30] R. Canetti, "Security and composition of multiparty cryptographic protocols," *J. CRYPTOLOGY*, vol. 13, pp. 143–202, 2000.

- [31] Y. Lindell, "How to simulate it—a tutorial on the simulation proof technique," *Tut. Found. Cryptogr.: Dedicated Oded Goldreich*, pp. 277–346, 2017.
- [32] D. Rathee et al., "Cryptflow2: Practical 2-party secure inference," in *Proc. 2020 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2020, pp. 325–342.
- [33] G. Oded, "Foundations of cryptography: Volume 2, basic applications," *Cambridge Univ. Press*, 2009.
- [34] P. Mohassel and Y. Zhang, "Secureml: A system for scalable privacy-preserving machine learning," in *Proc. IEEE 2017 Symp. Secur. Privacy*, 2017, pp. 19–38.
- [35] R. Canetti, "Universally composable security: A new paradigm for cryptographic protocols," in *Proc. IEEE 42nd Symp. Found. Comput. Sci.*, 2001, pp. 136–145.
- [36] R. Canetti, A. Cohen, and Y. Lindell, "A simpler variant of universally composable security for standard multiparty computation," in *Proc. Annu. Cryptol. Conf.*, 2015, pp. 3–22.
- [37] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.
- [38] A. Krizhevsky et al., "Learning multiple layers of features from tiny images," 2009.



Ruoli Zhao received the B.S. degree in computer science and the M.S. degree from Qinghai University, Qinghai, China, in 2021 and 2024, respectively. His research interests include privacy-preserving machine learning, cryptography and secure multi-party computation.



Yong Xie received the Ph.D. degree in computer science from Wuhan University, Wuhan, China, in 2016. He is currently a Professor with the School of Computer and Information Science, Qinghai Institute of Technology, Xining, China. His research interests include network protocol, protocol security, and privacy-preserving machine learning.



Debiao He (Member, IEEE) received the Ph.D. degree in applied mathematics from the School of Mathematics and Statistics, Wuhan University, Wuhan, China, in 2009. He is currently a Professor with the School of Cyber Science and Engineering, Wuhan University. He has authored or coauthored 100 research papers in refereed international journals and conferences, such as *IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING*, *IEEE TRANSACTIONS ON INFORMATION SECURITY AND FORENSIC*, and *Usenix Security Symposium*. His work has been cited more than 7000 times at Google Scholar. His main research interests include cryptography and information security, in particular, cryptographic protocols. He was the recipient of the 2018 IEEE Systems Journal Best Paper Award and 2019 IET Information Security Best Paper Award. He is on the Editorial Board of several international journals, such as *Journal of Information Security and Applications*, *Frontiers of Computer Science*, and *Human-centric Computing & Information Sciences*.



Kim-Kwang Raymond Choo (Senior Member, IEEE) received the Ph.D. degree in information security from the Queensland University of Technology, Brisbane City, QLD, Australia, in 2006. He currently holds the Cloud Technology Endowed Professorship with The University of Texas at San Antonio, San Antonio, TX, USA.



Zoe L. Jiang received the Ph.D. degree from the University of Hong Kong, Hong Kong, in 2010. She is currently a Professor with the School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen, China. Her research interests include secure multi-party computation, secure outsourcing computation, and cloud security.